Signal Processing of Musical Instruments

AELP101, Semester 2, 2024-2025
IIT - Delhi, Abu Dhabi

Joel Jobi, Karthik Nambiar
Group 4



Project
Tuesday 6th May, 2025

Part I: Fourier Analysis and Additive Synthesis

Objective

This part is mostly *investigative*. Its objective is to deconstruct the waveforms captured from two stringed instruments, a Nylon Stringed Guitar and a Piano, to find out what makes the sound from each instrument unique even when they play the same (fundamental) frequency. To do this, we intend to find out the distribution of the amplitudes of the overtones, i.e, how much of the sound consists of the fundamental, the first harmonic, the fifth harmonic, etc. We also investigate how this distribution evolves with time.[1]

At the end of this experiment, we would like to explain the sound produced from each instrument analyzed, either qualitatively or quantitatively[2] depending on the extent of analysis. As a sanity check, we would use the modeled waveforms to recreate the original sound, to test how well the model captures the sound of the instrument. (Additive Synthesis)

To obtain frequency information from the recorded signals, we use the Fast Fourier Transform (FFT) algorithm, and to observe how it changes with time, we use the windowing technique, ensuring each window captures a complete wave from node to node.[3]

Apparatus Used

- Nylon Stringed Guitar[4]
- Digital Instruments[5]
- 1/4" Instrument Cable
- Connecting Clips

Theory

Mechanical Standing Waves

A Standing Wave is a wave whose peaks and crests do not travel through space. They often occur due to interference between a disturbance and its reflection, and the waves thus set up follow certain boundary conditions depending on the rigid and elastic parts of the oscillator.

On a string fixed at its endpoints

For this case, only those disturbances survive which result in nodes (stationary points) at each endpoint. The differential equation for this case (assuming no damping) is as follows:

$$\frac{\partial^2 y}{\partial t^2} = \frac{T}{\mu} \cdot \frac{\partial^2 y}{\partial x^2}$$

With the boundary conditions that force the ends to be nodes:

$$y(0,t) = 0$$
$$y(L,t) = 0$$

Where y is the perpendicular distance from the length axis (height), x the distance along the string measured from one fixed end, T the tension in the string, μ the linear mass density, and L the length of the string or equivalently the distance between the fixed points.

Now we introduce a constant $\frac{1}{v^2} = \frac{T}{\mu}$, which does not have any direct relation to the standing wave, but is the

[1] For example, the higher harmonics might initially dominate, but the fundamental might be the longest lived.
[2] By modelling the waveform as a sum of decaying sinusoidal functions
[3] For amplitude information, we use the Hann window to reduce the effects of an incomplete signal, but for phase information, measured separately, we use a rectangular window.
[4] Enya Nova GO SP1
[5] Prerecorded, Soundfont source

speed of transmission of the waves that interfered to give rise to it. The solutions to this system are a family of standing waves (The fundamental and its harmonics), characterized by the parameter n, which takes integer values.

$$y_n(x,t) = 2A \sin\left(\frac{n\pi x}{L}\right) \cos\left(\frac{n\pi vt}{L}\right)$$

Where A is the amplitude of the parent traveling wave, or equivalently half the amplitude of the standing wave.

For stringed instruments like the Guitar and Ukulele, the string is in a superposition of these solutions, i.e, $Y(x,t) = a_1y_1 + a_2y_2 + a_3y_3 \cdots$, where a_n is the relative strength of each harmonic.

This is infact more general and applies to all instruments; The frequency heard is that of the fundamental, the unique sound or timbre of the instrument comes from the strength of each overtone, i.e, the specific values of a_n.[6]

Decay of Mechanical Standing Waves

Various factors contribute to the decay of a mechanical wave on a string. This includes energy losses due to elastic hysteresis (proportional to strain rate, $\frac{d\epsilon}{dt}$) air drag, and how well the acoustic body of the instrument supports oscillations in the string.

For small oscillations, these can be assumed to be proportional to oscillation velocity, which implies that the decay would be exponential, similar to that of underdamped oscillations in a spring mass system.

We use a falling exponential and a sinusoidal term to represent each harmonic, and attempt to find the parameters A_i, k_i, f_i, ϕ_i experimentally, for up to 18 overtones (maximum). Therefore the signal $S(t)$ can be represented as:

$$S(t) = \sum_{i=1}^{18} Ae^{-k_it} \sin\left(2\pi f_it + \phi_i\right)$$

Theoretical Distribution of the amplitudes of the harmonics.

The initial amplitudes of each harmonic can be theoretically calculated as a function of the position where the string is plucked/struck. This relation[source] is as follows:

$$a_n = \frac{2AL^2}{\pi^2 l(L-l)n^2} \cdot \sin\left(\frac{n\pi l}{L}\right)$$

Where A is the initial displacement involved in plucking the string, l is the distance from either end, and n stands for the nth overtone ($n = 0$ is the fundamental).

Fourier Analysis

An important theorem used in signal processing states that:

Fourier's Theorem. *Any continuous periodic function $f(x)$ can be expressed as a sum of sinusoids with appropriate phase and amplitude.*

A Fourier Transform takes a signal and gives the amplitudes, phases and frequencies of the sinusoidal waves which can be summed to recreate the signal. When implemented on digital signals which are discrete, the FFT or Fast-Fourier Transform algorithm is used. A Fourier transform can also be applied on non-periodic signals, on a restricted domain. The resultant Fourier series perfectly recreates the signal on that domain, but outside it, it merely repeats again.

The images on the next page illustrate how a square wave can be constructed from its Fourier Series, as well as the Fourier series of some well known periodic functions.

[6] For certain instruments , the overtones may be anharmonic, i.e. the ratio of the frequencies of the overtone to the fundamental may not be integral. For example, in a kalimba which can be modelled as a cantilever clamped at its fixed end, the transverse waves thus produced have the following nonlinear relation between ω and k

$$\omega = \sqrt{\frac{EI_y}{\rho A}} \cdot k^2$$

Where E is the Young's modulus of the tine, I_y is the second moment of area along the y-axis $\left(\int_A z^2 dA\right)$. This means that the ratio $\frac{\omega}{k}$, or the wave velocity, now depends on the frequency as well. In this case, even if the wavelengths of the overtones follow a harmonic series, $\frac{\lambda}{4}, \frac{3\lambda}{4}, \frac{5\lambda}{4}, \cdots$ the frequencies will not, as the wave velocities are now frequency dependent.

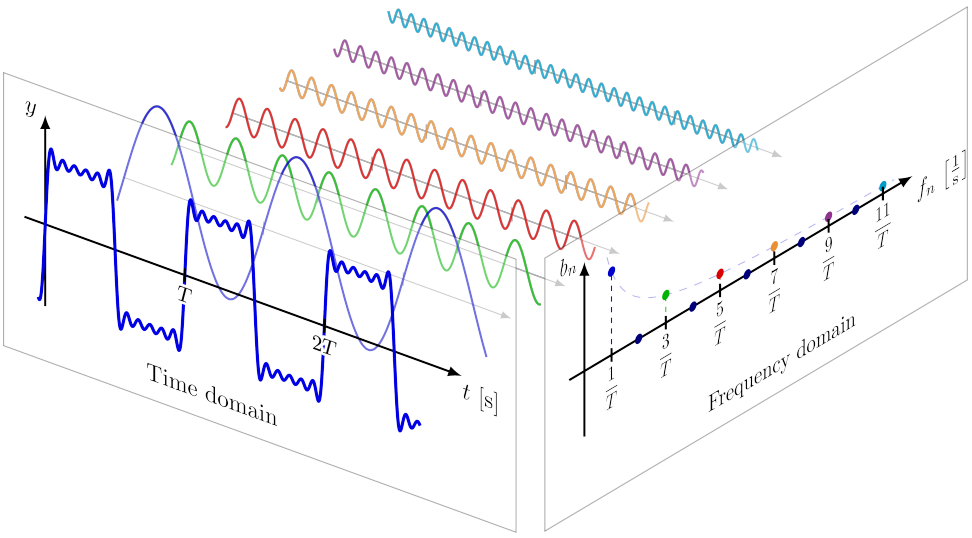


Figure 1: Fourier Synthesis of a Square Wave. *Left*: Time Domain, *Right*: Frequency Domain. A square wave only consists of odd harmonics

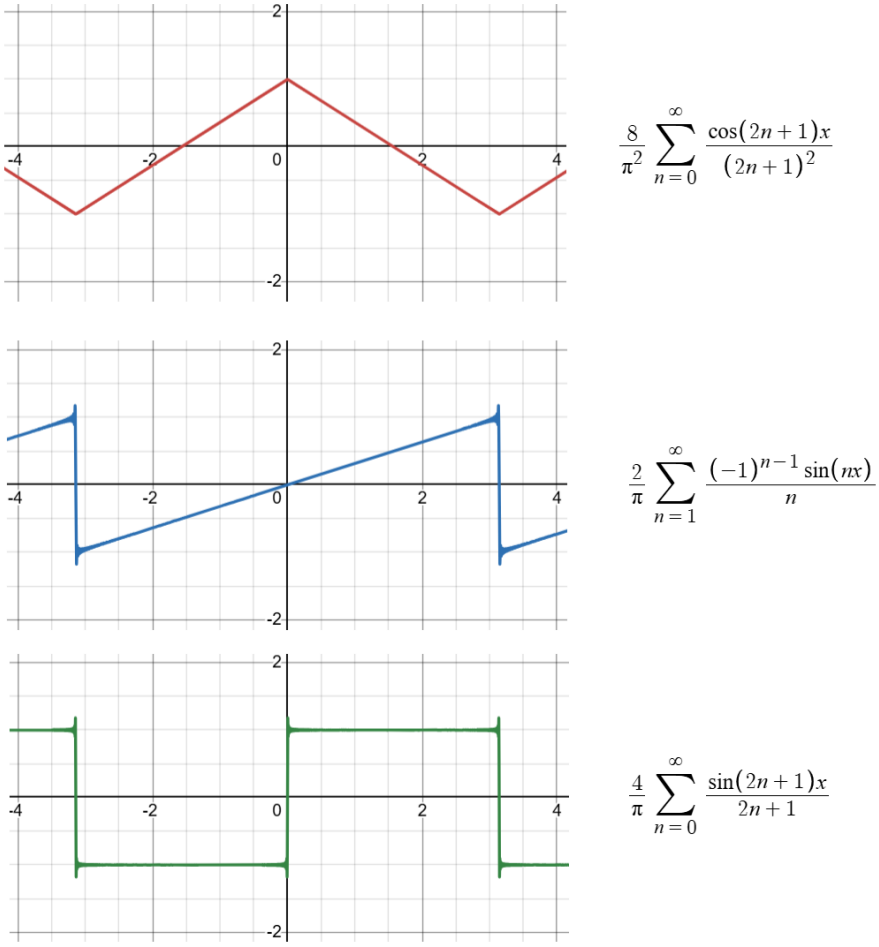


Figure 2: A Triangle, Sawtooth, and Square Wave of Unit Amplitude

Experimental Setup

We first record the signal (A single note of a known fundamental frequency) from the instrument into a .wav file, and then use the following code to obtain frequency information from it. The logic of the code is explained

as follows:

- We work with the assumption that the instruments are harmonic, which means that all the overtones are integer multiples of the fundamental frequency.[7]
- The period of the entire signal (which is a sum of other periodic signals) is the lowest common multiple of all their time periods, and because of this assumption, it is the fundamental's time period.
- Now, since the signal is not strictly periodic and its amplitude decays with time, we take a small section of the signal (a window) at a time, and perform an FFT here by assuming that the signal is relatively constant.[8]
- We then plot the amplitude of each frequency obtained in the FFT, with the timestamp of the center of the window. Doing this for the entire signal results in a table with the amplitudes of each harmonic at each timestamp.
- We then use least squares regression to fit the discrete points to the modeling equation, and hence find the values of the parameters.
- Another program then constructs a signal of the required duration by taking these parameters as input. We then compare the sound of the input waveform and the reconstructed one.

Windowed FFT algorithm

```
import numpy as np
import scipy.io.wavfile as wav
import scipy.signal
import csv
from scipy.fft import fft
import os

def get_dominant_peaks(signal_segment, fs, num_modes=18, min_freq=100, max_freq=6000):
    """Extract top N spectral peaks with frequency and amplitude."""
    N = len(signal_segment)
    windowed = signal_segment * np.hanning(N)
    spectrum = np.abs(fft(windowed))[:N//2]
    freqs = np.fft.fftfreq(N, d=1/fs)[:N//2]

    # Restrict to a frequency band
    mask = (freqs >= min_freq) & (freqs <= max_freq)
    spectrum = spectrum[mask]
    freqs = freqs[mask]

    # Find all local maxima
    peak_indices, _ = scipy.signal.find_peaks(spectrum, distance=5)
    peak_mags = spectrum[peak_indices]
    peak_freqs = freqs[peak_indices]

    # Take top N peaks
    top_indices = np.argsort(peak_mags)[-num_modes:][::-1]  # Descending order
    top_freqs = peak_freqs[top_indices]
    top_amps = peak_mags[top_indices]

    # Sort by frequency (optional)
    sort_order = np.argsort(top_freqs)
    return top_amps[sort_order], top_freqs[sort_order]
```

[7] This assumption holds very well for those instruments where string oscillations produce sound, such as the Bass, Guitar, Ukulele, and Piano. Instruments like the Xylophone, Kalimba, etc. which have oscillating metal plates, have overtones that are anharmonic; the solutions to their respective differential equations are not characterized by integer parameters.

[8] The length of the window is the time taken for 10-15 cycles of the waveform, (which is equal the same number of cycles of the fundamental, for harmonic overtones)

Windowed-FFT Algorithm (Continued)

```
def analyze_wav(file_path, num_modes=18, window_length_sec=0.07644675,
 hop_fraction=0.25, output_csv=None):

    fs, signal = wav.read(file_path)

    if output_csv is None:
        base_name = os.path.splitext(os.path.basename(file_path))[0]
        folder_name = os.path.basename(os.path.dirname(file_path))
        output_csv = f"{folder_name}_{base_name}.csv"

    if signal.ndim > 1:
        signal = signal.mean(axis=1)

    signal = signal / np.max(np.abs(signal))

    window_length = int(fs * window_length_sec)
    hop_size = int(window_length * hop_fraction)

    results = []

    for start in range(0, len(signal) - window_length, hop_size):
        segment = signal[start:start + window_length]
        center_time = (start + window_length // 2) / fs

        amps, freqs = get_dominant_peaks(segment, fs, num_modes)
        row = [center_time] + list(amps) + list(freqs)
        results.append(row)

    amp_headers = [f"Amp_{i+1}" for i in range(num_modes)]
    freq_headers = [f"Freq_{i+1}" for i in range(num_modes)]
    header = ["Time (s)"] + amp_headers + freq_headers

    with open(output_csv, "w", newline="") as f:
        writer = csv.writer(f)
        writer.writerow(header)
        writer.writerows(results)

    print(f"Saved modal analysis to: {os.path.abspath(output_csv)}")

analyze_wav("Nylon/C3.wav")
```

To use the above code, call the function with:

`analyze_wav("{filename/path}")`

It takes a .wav file as input, and outputs a .csv file with the amplitudes and frequencies. The following parameters in the *analyze_wav* and *get_dominant_peaks* function modify the FFT algorithm:

- *num_modes* : The number of modes obtained. Present in both functions
- *min_freq* : The minimum frequency above which the program searches for peaks in the raw FFT output
- *max_freq* : The maximum frequency above which the program searches for peaks in the raw FFT output
- *window_length_sec* : The duration of the window in seconds. Keep this as an integer multiple of the time period of the fundamental, in the example it is set to 10 times the time period of the note C3.
- *hop_fraction* : The time by which the window moves, as a fraction of the window duration. For example, 0.5 means that adjacent windows have 50% overlap, 0.25 means 75% and 1 means the windows have no overlap. Keep this between 0 and 1, and make sure that the product of the hop fraction and number of cycles in a window is an integer, so that the windows always begin and end with the cycles.

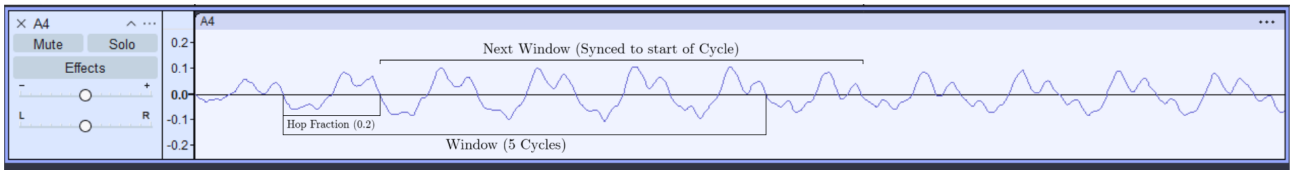


Figure 3: Illustration of recommended window length and hop fraction, visualized on Audacity

The below code takes the output of the first program and arranges the data in an easier to process structure.

CSV Cleanup Code

```
import csv
import os

def cleanup_modal_csv(input_csv):
    # Generate output filename by appending "_clean" before .csv
    base, _ = os.path.splitext(input_csv)
    output_csv = base + "_clean.csv"

    with open(input_csv, newline='') as f:
        reader = list(csv.reader(f))

    header = reader[0]
    num_modes = (len(header) - 1) // 2

    # Step 1: Collect all unique frequencies across rows
    all_freqs = set()
    data_by_time = []

    for row in reader[1:]:
        time = row[0]
        amps = row[1:1 + num_modes]
        freqs = row[1 + num_modes:]
        freq_amp_map = {float(freq): amp for amp, freq in zip(amps, freqs)}
        all_freqs.update(freq_amp_map.keys())
        data_by_time.append((time, freq_amp_map))

    # Step 2: Sort all frequencies in ascending order
    sorted_freqs = sorted(all_freqs)

    # Step 3: Write output CSV with unified frequency columns
    with open(output_csv, "w", newline='') as f:
        writer = csv.writer(f)
        writer.writerow(["Time (s)"] + [f"{freq:.3f}" for freq in sorted_freqs])

        for time, freq_amp_map in data_by_time:
            row = [time]
            for freq in sorted_freqs:
                row.append(freq_amp_map.get(freq, ""))
            writer.writerow(row)

    print(f"Saved cleaned CSV to: {output_csv}")

cleanup_modal_csv("Piano_C5.csv")
```

Signal Constructor

```
import numpy as np
from scipy.io.wavfile import write

# Sample rate and duration
sample_rate = 44100
duration = 5.0  # seconds
t = np.linspace(0, duration, int(sample_rate * duration), endpoint=False)
r=220 #fundamental frequency
# Define a list of expsines with parameters (A, k, f, phi)
expsines = [
(249.00129, 2.63227, 1*r,0),(128.59441, 2.38308, 2*r,0),
(6.66523, 2.62981, 3*r,0),(46.60635, 2.99151, 4*r,0),
(10.64565, 3.40945, 5*r,0),(4.93575, 4.72225, 6*r,0),
(9.51669, 3.38721 , 7*r,0),(4.299, 5.31225  , 8*r,0),
(8.53218, 13.66037, 9*r,0),(2.57448, 4.15758, 10*r,0),
(6.25496, 7.56688, 11*r,0),(1.79722, 4.68457 , 12*r,0),
(1.19145, 4.77753 , 13*r,0),(1.94734, 4.28449 , 14*r,0),
(1.83891, 6.76555, 15*r,0),(1.87871, 7.54312 , 16*r,0),
(5.1243, 13.04117, 17*r,0),(6.23217, 17.61705, 18*r,0)
]

# Generate the sum of all expsines
signal = np.zeros_like(t)
for A, k, f, phi in expsines:
    signal += A * np.exp(-k * t) * np.sin(2 * np.pi * f * t + phi)

# Normalize to 16-bit range
signal /= np.max(np.abs(signal))
signal = (signal * 32767).astype(np.int16)

# Save to WAV file
write("expsines.wav", sample_rate, signal)
```

This program has the following tunable parameters.

- *duration*: The required duration of the output signal
- r: Fundamental frequency
- *expsines*: The distribution of amplitudes, decay constants, frequency ratios and phase differences across the overtones

---

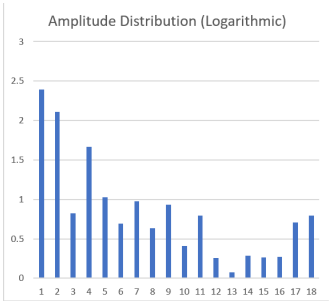
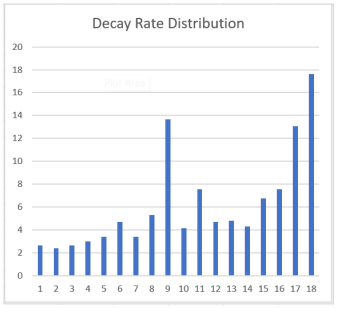
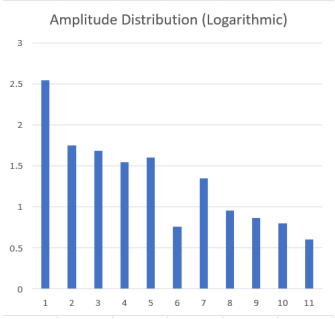
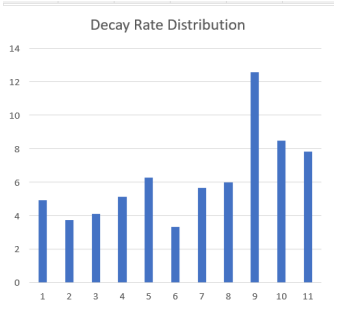
Phase Considerations

While collecting the data for this experiment, we have decided to deliberately ignore collecting phase information, for the following reasons.

- While the relative phase between the component signals do matter in deciding the shape of the waveform, the human ear is relatively insensitive to such phase differences unless they cause a major change in amplitude.
- For stringed instruments, the initial phases of each harmonic also depend on the location where the string was plucked, so we would need to record phase for each sample analyzed, instead of just once per instrument.
- In addition to the above, phase differences also change with time, i.e, the phase is modulated due to reasons such as non-ideal boundary conditions (The bridge and nut are not perfectly rigid), energy exchange between harmonics, etc. which complicates measurement of phase.

Observations

Table 1: The amplitude and decay rate distributions for the guitar (top) and piano (below)

B3.wav	246.94Hz			B3_reconstructed.wav
B4.wav	493.88Hz			B4_reconstructed.wav

So, the reconstructed waveforms can be expressed mathematically as:

$$\begin{aligned}\text{B3}(t) = {} & 10.00e^{-2.632t}\cos\omega t + 5.164e^{-2.383t}\cos 2\omega t + 0.268e^{-2.630t}\cos 3\omega t + 1.872e^{-2.992t}\cos 4\omega t \\ & + 0.428e^{-3.409t}\cos 5\omega t + 0.198e^{-4.722t}\cos 6\omega t + 0.382e^{-3.387t}\cos 7\omega t + 0.173e^{-5.312t}\cos 8\omega t \\ & + 0.343e^{-13.66t}\cos 9\omega t + 0.103e^{-4.158t}\cos 10\omega t + 0.251e^{-7.567t}\cos 11\omega t + 0.072e^{-4.685t}\cos 12\omega t \\ & + 0.048e^{-4.778t}\cos 13\omega t + 0.078e^{-3.387t}\cos 14\omega t + 0.074e^{-6.766t}\cos 15\omega t + 0.075e^{-7.543t}\cos 16\omega t \\ & + 0.206e^{-13.041t}\cos 17\omega t + 0.25e^{-17.617t}\cos 18\omega t\end{aligned}$$

Where $\omega = 39.3$ rads^{-}1.

$$\begin{aligned}\text{B4}(t) = {} & 10.00e^{-4.926t}\cos\omega t + 1.592e^{-3.752t}\cos 2\omega t + 1.384e^{-4.092t}\cos 3\omega t + 1.001e^{-5.127t}\cos 4\omega t \\ & + 1.147e^{-6.273t}\cos 5\omega t + 0.165e^{-3.334t}\cos 6\omega t + 0.632e^{-5.651t}\cos 7\omega t + 0.259e^{-5.983t}\cos 8\omega t \\ & + 0.212e^{-12.588t}\cos 9\omega t + 0.18e^{-8.466t}\cos 10\omega t + 0.114e^{-7.847t}\cos 11\omega t\end{aligned}$$

Where $\omega = 78.6$ rads^{-}1.

Qualitative analysis of other instruments

Violin

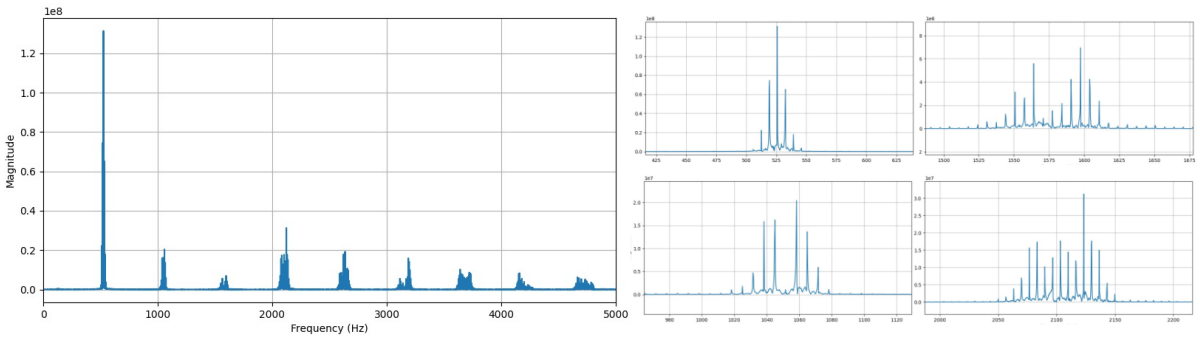


Figure 4: The FFT for a violin note (C5) where the first 4 peaks are zoomed into to show the spread in frequencies.

The Waveform from a violin exhibits the following properties.

- Since a violin is continuously bowed, it is an example of driven oscillation. Hence it does not exhibit decay.
- A violin exhibits frequency modulation, i.e the frequency also oscillates with time with a low frequency. This is the reason for the 'vibrato' like feel of violins and other bowed instruments, and explains the spread of frequency at each harmonic.

Concluding Remarks and Applications

- The amplitude distribution among the harmonics, and their respective decay rates, was found for the notes B3 and B4, on the Nylon String and Piano respectively.

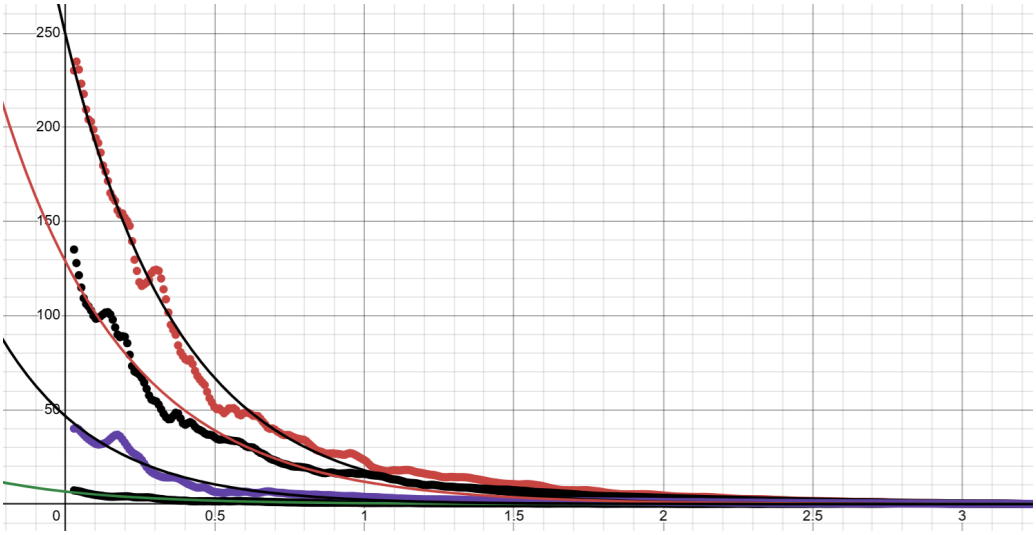


Figure 5: A visualization of the time-evolution of the amplitudes of the first 4 harmonics, and their best-fit exponential decay curves. *Exponential Regression by Desmos.*

- It was found out that the decay rates have a roughly increasing trend with increasing harmonics, but more harmonic data must be collected to model a relation
- Their sounds were reconstructed faithfully, and the program written can now be used to generate tones of any frequency for these instruments.
- The tone generating program is more general, and if the amplitude and decay distribution is known, as well as the individual frequency ratios (for inharmonic instruments) and phase relations, then the sound of an instrument can be accurately reproduced across all frequencies
- The decaying sinusoid model employed here can model a large variety of instruments, but lacks any modulation beyond the exponential decay envelope. The next part introduces circuits which can alter the waveform captured from an instrument in real-time.
- An interesting application of the above work is in finding the amplitude and decay distributions for two instruments, and averaging them in some manner so as to combine the properties of both the instruments. For example, the sound of a grand piano, combined with the long sustain of a violin
- Fourier Analysis is used in many similar contexts, one common example being in changing the speed of an audio clip without affecting pitch and vice versa
- Collected data for some other notes on the piano and nylon string guitar, are available here. Due to time constraints, it is not represented in the observations section.

References

- Standing Waves
- Audacity : *For audio trimming, manual time period measurement and stereo to mono conversion.*
- Amplitude Distribution Derivation
- Natural modes of a cantilever: Derivation : A Kalimba is modelled as a cantilever. This explains the origin of the inharmonic overtones in this instrument.

Part II: Audio Effects - Distortion

Objective

The objective of this part is to design circuits which can shape the tone from an instrument to our liking, in real-time as we play. There are many effects that one can apply to an analog signal, such as distortion, tremolo, octave up/down, reverb, delay, EQ, etc. In this section, we will build a distortion pedal, and simulate its output using a circuit simulation software (Falstad).

Apparatus Used

- Operational Amplifiers[9]
- Diodes[10]
- Resistors
- Breadboard
- Connecting Wires
- AUX Cable
- Speaker[11]

Theory

Guitar pedals are electronic devices used to modify the sound of an electric guitar (sometimes other instruments too), typically placed in-line between the guitar and the amplifier. These pedals introduce a variety of effects by altering the audio signal in either analog or digital domains, or both. The main categories of effects include distortion-based, time-based, modulation, dynamic modification and equalizers. Some of these are explained below.



Figure 6: A number of pedals in series. Each pedal modifies the signal of the output of the previous one.

[9] LM741
[10] 1N4007
[11] JBL GO

Pedal Categories

Distortion Pedals: These are among the most common types of effects, including overdrive, distortion, and fuzz,[12]. They work by clipping the audio signal, either by soft clipping in the case of overdrive or hard clipping in distortion and fuzz to produce harmonic content that adds 'grit' or 'crunch' to the sound. Implementation typically involves transistor or op-amp circuits with diodes in feedback loops to control the clipping behavior.

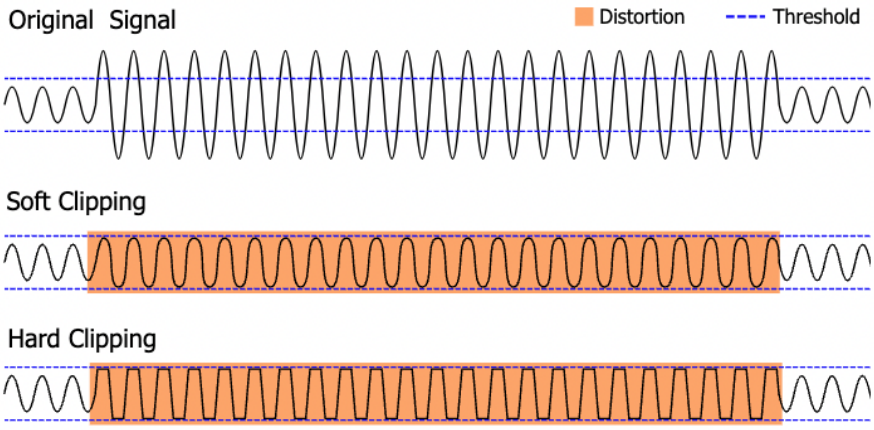


Figure 7: *The difference between soft and hard clipping*: The signal is amplified so it crosses the diode's threshold, and the diode clips the signal based on its characteristic behavior. For example, when blue LEDs are used as the diodes, they produce extremely strong distortion.

Dynamic Effects: These include compressors, limiters, and noise gates. Compressors reduce the dynamic range of the signal, making quieter parts louder and louder parts quieter, thereby smoothing out volume inconsistencies. Limiters put a hard limit on the volume. They are implemented using variable gain control mechanisms such as voltage-controlled amplifiers (VCAs) or digital envelopes.

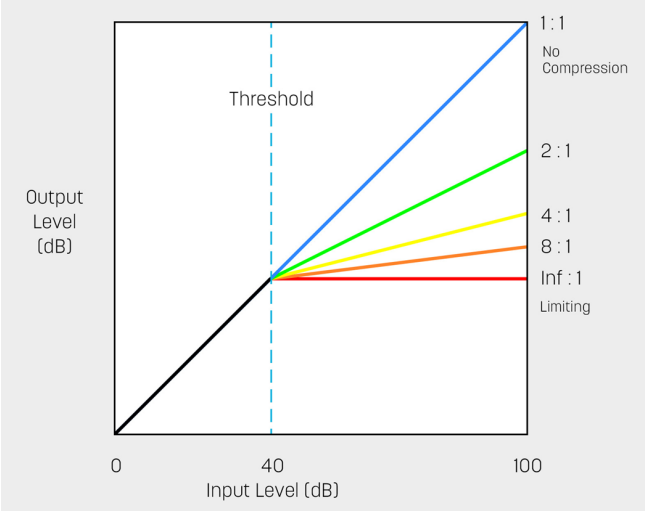


Figure 8: The Input/Output Curve for compressors and limiters.

Equalization and Filter Pedals: These devices modify the frequency content of the signal. Graphic and parametric EQ pedals allow selective boosting or cutting of certain frequency bands, usually implemented through cascaded filter circuits. Wah pedals are a special case, sweeping a band-pass filter across the frequency spectrum based on foot pedal position.

[12] in increasing order of distortion applied to the signal

Experimental Setup

This circuit schematic (Falstad file) can be downloaded here. The circuit consists of a preamplifier to bring the input voltage to above the threshold voltage of the diodes[13], and the clipper section provides distortion by hard-clipping the signal.

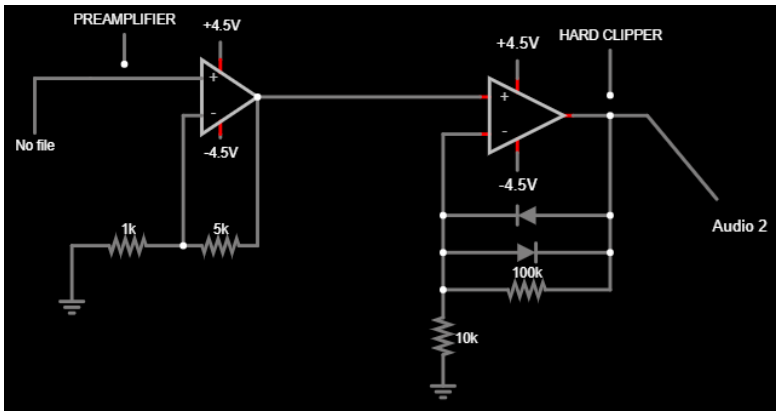


Figure 9: Circuit Diagram (The 2 free nodes are labels, and are not part of the circuit)

Observations

Here is an example of an input waveform, and here is its output, simulated on Falstad.

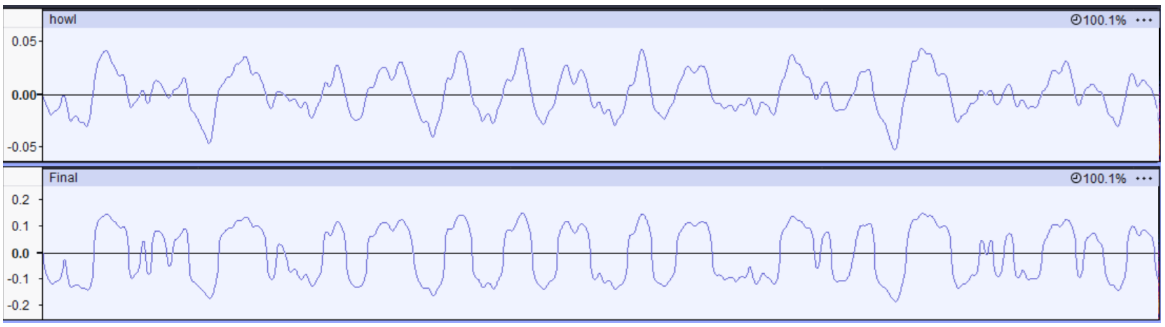


Figure 10: A comparison of input and output (bottom) waveforms. Since the output has been considerably amplified, it is shown on a different scale.

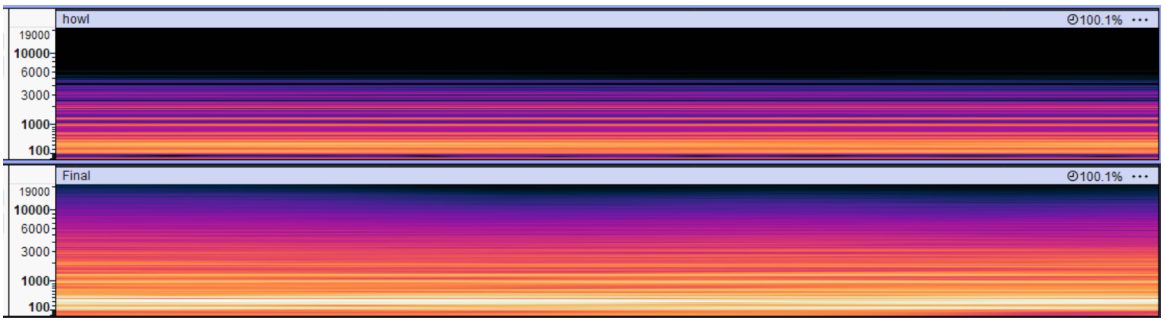


Figure 11: A spectrogram visualization of the additional overtones and subharmonics added by the distortion.

---

[13] The input voltage of the line in is of the order of ≈500mV

Concluding Remarks

- A Distortion pedal was successfully constructed and utilized.
- The Sampled audio and extracted data are available here